

tokenToPay and exerciseFees Causes forceExercise to Revert

Summary

In the `forceExercise` flow, two fundamentally different calculations are used: `exercise()` burns shares based on `tokenToPay` (which includes accumulated premium), while `getExerciseDeltas()` calculates refunds based on `exerciseFees` (geometry-based, ignoring premium). When premium debt is significant, this mismatch causes `revoke()` to underflow and revert, making force exercise impossible. Combined with the zombie account vulnerability (H-01), this creates positions that are **permanently inescapable**.

Vulnerability Detail

The Two Calculations

CALCULATION 1: **TOKENTOPAY** (IN **EXERCISE()**)

```
// CollateralTracker.sol - exercise()
tokenToPay = swappedAmount - (longAmount - shortAmount) - realizedPr
//
//
```

CALCULATION 2: EXERCISEFEES (IN EXERCISECOST())

```
// CollateralTracker.sol - exerciseCost()
exerciseFees = pricePerContract * EXERCISE_FEE_BPS / 10000;
//          ^^^^^^^^^^^^^^^^^^^^^
//          GEOMETRY-BASED ONLY, NO PREMIUM
```

The Flow That Breaks

```
forceExercise() {
  1. delegate(account)           // Adds 2^248-1 to balanceOf

  2. exerciseFees = exerciseCost() // Geometry-based fee

  3. _burnOptions()              // Calls exercise()
    ↳ exercise()
      ↳ tokenToPay = ... - realizedPremium // INCLUDES PREMIUM
      ↳ BURNS shares based on tokenToPay   // MORE than exerciseFees

  4. getExerciseDeltas()         // Uses exerciseFees
    ↳ Calculates refund based on exerciseFees // IGNORES extra burn

  5. refund()                    // Refunds based on exerciseFees
                                // INSUFFICIENT to cover tokenToPay

  6. revoke(account)             // Tries to subtract 2^248-1
    ↳ UNDERFLOW!                 // Balance < 2^248-1 after insuffi
    ↳ REVERT

}
```

Concrete Example

```

Initial State:
- User real shares: 200 ETH equivalent
- After delegate(): balance = 200 + 2^248-1

Step 2 - exercise():
- swappedAmount: -100 ETH (ITM swap, user receives)
- shortAmount: 500 ETH
- realizedPremium: -300 ETH (OWES 300 ETH premium)
- tokenToPay = -100 - (0 - 500) - (-300) = 700 ETH

Shares BURNED: 700 ETH worth
Balance after: 2^248-1 + 200 - 700 = 2^248-1 - 500

Step 3 - exerciseFees:
- exerciseFees: -50 ETH (geometry-based only)

Step 4 - refund():
- Refund amount: 50 ETH (based on exerciseFees)
- Balance after: 2^248-1 - 500 + 50 = 2^248-1 - 450

Step 5 - revoke():
- Tries to subtract: 2^248-1
- Current balance: 2^248-1 - 450
- Result: -450 ETH

UNDERFLOW! Transaction REVERTS!

```

Impact

Direct Impact

CONDITION	RESULT
Position has premium debt	Force exercise reverts
Position in zombie zone (H-01)	Cannot close normally
Combined effect	PERMANENT FUND LOCK

Attack Scenario

1. Setup Phase:

- Attacker opens long position with minimal collateral

- Position accumulates significant premium debt over time

2. Trap Phase:

- Price moves adversely, putting account in zombie zone
- Account cannot close (fails solvency check)
- Account cannot be liquidated (passes solvency check)

3. Permanent Lock:

- Force exerciser attempts to free the position
- `tokenToPay` includes massive premium debt
- `exerciseFees` only covers geometry
- `revoke()` underflows → reverts
- **Position is trapped forever**

Systemic Risk

- Locked positions continue holding Uniswap liquidity
- Premium accumulation continues indefinitely
- Protocol has no mechanism to resolve these positions
- Eventually could lock significant protocol TVL

Proof of Concept

Core Demonstration

```

function test_RevokeUnderflow_Scenario() public {
    // Initial state
    uint256 realShares = 200 ether;
    uint248 DELEGATION = type(uint248).max;

    // After delegate()
    uint256 afterDelegate = realShares + uint256(DELEGATION);

    // exercise() with premium debt
    int256 tokenToPay = -100 ether - (0 - 500 ether) - (-300 ether);
    // tokenToPay = 700 ETH (must pay due to premium debt)

    uint256 sharesToBurn = uint256(tokenToPay); // 700 ETH
    uint256 afterExercise = afterDelegate - sharesToBurn;

    // exerciseFees (geometry-based)
    int256 exerciseFees = -50 ether; // Much smaller!

    // refund() based on exerciseFees
    uint256 refundAmount = 50 ether;
    uint256 afterRefund = afterExercise + refundAmount;

    // revoke() tries to subtract DELEGATION
    int256 netChange = int256(realShares) - int256(sharesToBurn) + i
    // netChange = 200 - 700 + 50 = -450 ETH

    // UNDERFLOW! afterRefund < DELEGATION
    assertTrue(netChange < 0, "Underflow will occur!");
}

```

Test Output

≡ H-03: revoke() Underflow Scenario ≡

Step 0: Initial real shares: 200 ETH equivalent

Step 1: After delegate()

Balance: $\text{realShares} + 2^{248}-1$

Step 2: exercise() with premium debt

swappedAmount: -100 ETH (ITM swap)

shortAmount: 500 ETH

```
realizedPremium: -300 ETH (OWES)
```

tokenToPay: 700 ETH (MUST PAY)

Shares BURNED: 700

Balance after exercise: $\sim 2^{248} - 700e18$

Step 3: exerciseFees from exerciseCost()

exerciseFees: -50 ETH (geometry only)

Step 4: `getExerciseDeltas()`

Expected to cover: 50 ETH (exerciseFees)

Actually burned: 700 ETH (tokenToPay)

Unaccounted: 650 ETH

Step 5: refund()

Refund amount: 50 ETH

Step 6: revoke()

Trying to subtract: $2^{248}-1$

Net share change: NEGATIVE

-45000000000000000000

!!! UNDERFLOW WILL OCCUR !!!

After `revoke()`, balance would be negative by: 450 ETH

Result: forceExercise() REVERTS

Position cannot be force exercised → becomes permanent zombie

Run the PoC

```
FOUNDRY_PROFILE=poc forge test --match-contract H03_ForceExercise_Mi
```

Result: 5 tests passed

Tools Used

- Manual code review
- Foundry for PoC development
- Control flow analysis

Recommended Mitigation

Option 1: Align Calculations (Recommended)

```
function forceExercise(...) external {
    // Calculate ACTUAL tokenToPay BEFORE burning
    int256 actualTokenToPay = _calculateTokenToPay(account, tokenId)

    delegate(account);

    // Use actualTokenToPay for BOTH burn and refund calculations
    _burnOptions(...);

    // Pass actualTokenToPay instead of exerciseFees
    refundAmounts = PanopticMath.getExerciseDeltas(
        ...,
        actualTokenToPay, // Use actual amount, not geometry-based
        ...
    );

    // Now refund properly compensates for actual burn
    _refund(...);

    revoke(account); // Will not underflow
}
```

Option 2: Pre-Check Solvency

```

function forceExercise(...) external {
    // Calculate if force exercise will succeed
    int256 tokenToPay = _calculateTokenToPay(...);
    int256 exerciseFees = exerciseCost(...);

    // If gap would cause underflow, require additional collateral
    int256 gap = tokenToPay - exerciseFees;
    if (gap > int256(userBalance)) {
        revert InsufficientCollateralForExercise();
    }

    // Proceed with force exercise
    ...
}

```

Option 3: Include Premium in exerciseCost

```

function exerciseCost(...) internal returns (int256) {
    // Include accumulated premium in the cost calculation
    int256 geometryFee = pricePerContract * EXERCISE_FEE_BPS / 10000
    int256 accumulatedPremium = _getAccumulatedPremium(tokenId);

    return geometryFee + accumulatedPremium; // Aligned with tokenT
}

```

Critical Priority

This fix is essential as it creates a permanent DOS condition when combined with H-01. Positions with significant premium debt become completely inescapable, locking user funds and protocol liquidity indefinitely.

Relationship to Other Findings

This vulnerability **amplifies H-01 (Zombie Accounts)**:

- H-01 alone: Account stuck but potentially resolvable via force exercise
- H-01 + H-03: Account stuck with **no resolution mechanism**

The combination should be considered **CRITICAL** severity as it enables permanent, irreversible fund lock.